Academic Year: 2026-27
Semester: III
Class / Branch: SE IT
Subject: ADSA Lab
Name of Instructor: Prof. Sneha Dalvi

Name of Student: Kedar Vanare
Student ID:26204020
Date of Performance:08/09/26
Date of Submission:22/09/26

Experiment No. 1

Aim: To Implement Merge sort and Quicksort for the given list of integer values and find space and time complexity.

Code :

```
#include <stdio.h>

#include <stdlib.h>

// Merge two subarrays L and R into arr void

merge(int arr[], int left, int mid, int right) {  int

i, j, k;

 int n1 = mid - left + 1;

int n2 = right - mid;   //

Temporary arrays
```

```
 int *L = (int *)malloc(n1 * sizeof(int));
int *R = (int *)malloc(n2 * sizeof(int));
 // Copy data to temporary arrays L[] and R[]
 for (i = 0; i < n1; i++)
L[i] = arr[left + i];  for
(j = 0; j < n2; j++)
 R[j] = arr[mid + 1 + j];

 i = 0; j = 0; k = left;
while (i < n1 && j < n2) {
if (L[i] <= R[j])  arr[k++]
= L[i++];  else  arr[k++] =
R[j++];
 }

 while (i < n1)
arr[k++] = L[i++];
```

```
 while (j < n2)
arr[k++] = R[j++];


 free(L);
free(R); }


void mergeSort(int arr[], int left, int right) {
if (left < right) {  int mid = left + (right -
left) / 2;  // Sort first and second halves
mergeSort(arr, left, mid);  mergeSort(arr,
mid + 1, right);  // Merge the sorted halves
merge(arr, left, mid, right);
 }
}
```

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ gcc -o EXP1 EXP1.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ ./EXP1
Original array: 38 27 43 3 9 82 10
Sorted array using Quick Sort: 3 9 10 27 38 43 82
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ ~~
```

```
void printArray(int arr[], int size) {
for (int i = 0; i < size; i++)
printf("%d ", arr[i]);  printf("\n");
}

int main() {
 int arr[] = {38, 27, 43, 3, 9, 82, 10};  int size
= sizeof(arr) / sizeof(arr[0]);
printf("Original array: ");
printArray(arr, size);  mergeSort(arr, 0,
size - 1);  printf("Sorted array using Merge
Sort: ");  printArray(arr, size);  return 0;
}
```

Output-

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/SEITKV$ ./exp1
Original array: 38 27 43 3 9 82 10
Sorted array using Merge Sort: 3 9 10 27 38 43 82
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/SEITKV$ gedit exp1.c
```

A. Merge Sort

Definition: Merge Sort is a sorting algorithm based on the Divide and Conquer technique. It divides an array into smaller subarrays, sorts them, and merges them into a sorted array.

Algorithm:

1. Start with the given array.
2. Divide the array into two halves.
3. Recursively apply Merge Sort to both halves.
4. Compare elements of both sorted halves.
5. Merge them into a single sorted array.
6. Repeat until the entire array is sorted.
7. Stop.

Conclusion: Thus, Merge Sort was successfully implemented to sort the given array using the Divide and Conquer technique.

QUICK SORT:

```
#include <stdio.h>


void swap(int *a, int *b) {

int temp = *a;     *a = *b;

   *b = temp;

}
```

```
int partition(int arr[], int low, int high) {
int pivot = arr[high];     int i = low - 1;

   for (int j = low; j <= high - 1; j++) {
if (arr[j] <= pivot) {            i++;
swap(&arr[i], &arr[j]);
      }
   }

   swap(&arr[i + 1], &arr[high]);
return i + 1;
}

void quickSort(int arr[], int low, int high) {
if (low < high) {        int pi = partition(arr,
low, high);

      quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
   }
```

```
}

void printArray(int arr[], int size) {
for (int i = 0; i < size; i++) {
printf("%d ", arr[i]);
   }
   printf("\n");
}

int main() {
   int arr[] = {38, 27, 43, 3, 9, 82, 10};
int size = sizeof(arr) / sizeof(arr[0]);

   printf("Original array: ");
printArray(arr, size);

   quickSort(arr, 0, size - 1);

   printf("Sorted array using Quick Sort: ");
printArray(arr, size);
```

```
    return 0;
}
```

OUTPUT -

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ gcc -o EXP1 EXP1.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ ./EXP1
Original array: 38 27 43 3 9 82 10
Sorted array using Quick Sort: 3 9 10 27 38 43 82
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ ~~
```

B. Quick Sort

Definition: Quick Sort is a Divide and Conquer sorting algorithm that selects a pivot element and partitions the array so that smaller elements appear before the pivot and larger elements appear after it.

Algorithm:

1. Start with the given array.
2. Select the last element as the pivot.
3. Partition the array around the pivot.
4. Place the pivot in its correct position.
5. Recursively sort the left subarray.
6. Recursively sort the right subarray.
7. Stop when a subarray contains zero or one element.

Conclusion: Thus, Quick Sort was successfully implemented to sort the given array using partitioning and recursion.

Conclusion:

We successfully implemented and analyzed two fundamental sorting algorithms—Merge Sort and Quick Sort—on a given list of integers. Both algorithms follow the divide-and-conquer paradigm but differ significantly in their approaches and efficiency in terms of space and time.